Abstract

Course: DLBDSPBDM01 – Build a Data Mart in SQL; **Programme:** B.Sc. Computer Science; **Author:** Zubair Ahmed Khan; **Matriculation Number:** 92127983; **Tutor:** Prof. Dr. Anna Androvitsanea; **Submission Date:** 2026-07-29

Introduction

The short-term accommodation market has grown into a global digital ecosystem, with platforms such as Airbnb acting as intermediaries between property owners and travellers. The present work documents the design, implementation, and validation of a relational database, **AirbnbDB**, developed as a coursework deliverable for *Build a Data Mart in SQL*. The objective is a normalised, queryable data mart that mirrors the core operations of an Airbnb-style marketplace — from the moment a host publishes an accommodation until a guest leaves a review and any complaint is resolved. By structuring the data in MySQL, the project demonstrates how relational concepts (entities, primary and foreign keys, one-to-many, many-to-many, recursive and ternary relationships) can be applied to a real-world domain. The final artefact runs on any standard MySQL Server, populates itself with realistic dummy data, and supports join queries that simulate managerial reporting.

Methodology

Requirements were first gathered from the Airbnb use case and translated into business rules, roles, and actions. From these, a conceptual Entity–Relationship diagram was produced, identifying 21 entities together with one recursive relationship (Employee–Manager) and one ternary relationship (Booking–Payment–Reservation). The diagram was converted into a logical schema, decomposed through the first three normal forms to remove redundancy and prevent update, insertion, and deletion anomalies. Implementation took place in MySQL using MySQL Workbench: a single self-contained SQL script contains all `CREATE DATABASE`, `CREATE TABLE`, `ALTER TABLE`, and `INSERT INTO` statements, executed top-to-bottom to create the schema, enforce foreign-key constraints, and load dummy data. Three SQL test cases then verified referential integrity.

Database Functionality and Implementation

The mart is built around a small number of central entities. `User` stores all platform participants; `Guest` and `Host` specialise `User` through one-to-one foreign keys. `Accommodation` is owned by a `Host` and physically described by a `Place`, which references a `Location` and a set of `HouseRules`. `Booking` connects a `Guest` to an `Accommodation` for a date range, and is further linked to a `Payment` and a `Reservation` through the ternary association. Reviews, complaints, and notifications tie back to `User`, ensuring every interaction can be traced to a real participant. Supporting entities (`Amenity`, `AmenityListing`, `Media`, `Listing`, `Employee`, `Language`, `UserLanguage`, `Income`) enrich the model without breaking normalisation. The implementation consists of 21 tables, each declared with explicit `PRIMARY KEY` and `FOREIGN KEY` clauses; `ALTER TABLE` statements reinforce late-bound foreign keys (e.g. `PlaceID` in `Accommodation`, `PaymentDate` in `Payment`). Monetary values use `DECIMAL(10,2)`; `DATE`, `DATETIME`, and `VARCHAR` cover temporal and textual attributes. Consistency is maintained at three levels: schema (3NF-compliant), data (≥20 records per table), and integrity (foreign-key constraints reject orphan rows).

Testing

The database was validated using three SQL test cases executed against the live MySQL instance. **Test Case 1** retrieves the complete booking information for every reservation, joining `Booking` with `Guest`, `User`, and `Accommodation` to surface the guest name, accommodation title, and check-in and check-out dates. **Test Case 2** calculates host-level accommodation statistics and income by joining `Accommodation` with `Host` and `User` and aggregating the `TotalPrice` of all related bookings, exposing the total earnings of each host. **Test Case 3** retrieves the complete accommodation information, joining `Accommodation`, `Host`, and `User` to surface the host name alongside the property title, location, and pricing. Each test case uses multi-table joins that traverse the foreign-key network, providing empirical evidence that the relationships, the keys, and the dummy data are internally consistent.

Reflection

The original objectives — design a normalised data mart, implement it in MySQL, populate it with representative data, and demonstrate its usefulness through meaningful queries — were all met. The twenty-one-table schema captures the full booking lifecycle, the SQL script executes without errors, every table contains at least twenty records, and the three test cases return results that match expectations. The resulting data mart is scalable because new users, hosts, accommodations, and bookings can be inserted without any change to the schema, while data consistency is preserved by the foreign-key constraints and by adherence to normal forms. Possible future improvements include indexes on frequently joined columns, a stored-procedure layer for routine transactions, the migration of monetary data to a dedicated currency table, and a front-end dashboard that consumes the existing SQL queries through a REST interface. The project as a whole demonstrates that careful requirement analysis, a sound ER model, disciplined normalisation, and methodical testing together produce a database that is both academically rigorous and practically usable.